Product Requirements Document

Contract Query Assistant using Agentic RAG & Knowledge-Based RAG

DOMAIN
(LegalTech (Contract Intelligence Platform)
Contracts, Legal Research, Compliance

1.Problem statement

1.1Overview

The Contract Query Assistant is an AI-powered LegalTech solution that enables users to upload, search, and query contract documents using natural language. By leveraging Knowledge-Based RAG and Agentic RAG, it delivers accurate, context-aware answers with source citations, reducing manual contract review effort and improving information accessibility.

2.Project Type

Solution Type

- Conversational AI Chatbot
- Retrieval-Augmented Generation (RAG)
- Agentic AI System
- Document Intelligence Platform

Technology Stack

- Frontend: Streamlit
- Backend: FastAPI
- LLM: OpenAI GPT, Claude Models
- Vector Database: FAISS
- Agent Framework: LangGraph / LangChain
- Storage: Azure Blob Storage

3.Project Definition

Goals

1. Enable users to upload one or more contracts (PDF) and have them automatically processed into a searchable knowledge base.

2. Allow Users can ask natural-language questions about contract contents through a chat interface and receive accurate, grounded answers with source citations (document name, page/section).

3. Support an agentic reasoning path for complex queries that require decomposing the question, retrieving from multiple documents, and synthesizing a comparative answer.

4. Support a lightweight knowledge-based RAG path for direct, single-document factual questions, optimized for low latency.

5. Provide visibility into document ingestion status (uploaded, processing, indexed, failed) so users trust that their documents are ready to query.

6. Maintain session-based conversation history so users can ask follow-up questions in context.

7. Demonstrate a clean separation of concerns between a serverless ingestion pipeline (Azure Functions) and a synchronous query/agent API (FastAPI).

User Goals

- Ask questions about contracts in natural language.
- Receive accurate answers with supporting references.
- Search across multiple contracts simultaneously.
- Upload and manage contract documents easily.

Technical Goals

- Support scalable document ingestion.
- Provide low-latency responses (<5 seconds).
- Support multi-document retrieval.
- Maintain conversation history and session context.
- Ensure answer traceability through citations.

Non-Goals

- This system is not a substitute for legal advice or legal review; it surfaces and summarizes contract text but does not provide legal interpretation or risk judgments.

- The capstone scope does not include enterprise-grade access control, multi-tenant security, or role-based permissions — a single shared workspace is assumed.

- Real-time collaborative editing of contracts is out of scope; the system is read-only with respect to uploaded documents.

- Handling extremely large document sets (thousands of contracts) is out of scope; the design targets a small-to-moderate, user-managed collection sufficient to demonstrate the architecture, with horizontal scaling noted as future work rather than a tested requirement.

- Support for handwritten or heavily scanned/low-quality contracts (requiring advanced OCR correction) is a stretch goal, not a core

requirement.

- Multi-language contract support is out of scope for the initial version; English-language contracts are assumed.

Feature Definition

4.1 Frontend — Streamlit Application

The frontend is a two-screen Streamlit application.

Screen 1: Chat Interface

- Query input box for natural-language questions, with a send button and Enter-to-submit.

- Scrollable chat history showing the conversation turn by turn (user query → assistant response).

- Response display that renders the answer text along with cited source snippets (contract name + page/clause reference), expandable for full context.

- Visual indicator of which RAG path handled the query (e.g., a small badge: "Direct lookup" vs. "Agentic / multi-document") for transparency and demo purposes.

- Session management: ability to start a new chat session, and a session/document selector to scope questions to specific uploaded contracts or "all documents."

- Loading/typing indicator while the backend is processing a query.

- Basic error handling in the UI (e.g., "No relevant clause found" or "Backend unavailable, please retry").

Screen 2: Document Ingestion

- File upload widget supporting PDF with multi-file upload support.
- A document table/list showing: file name, upload timestamp, file size, and processing status (Queued → Processing → Completed / Failed).
- Status auto-refresh (polling) so users can watch a document move through the pipeline without manually reloading.
- Ability to view basic extracted metadata per document (e.g., page count, detected contract type if classified).
- Ability to remove/delete a document from the knowledge base.
- Clear error messaging if ingestion fails (e.g., unsupported format, corrupted file, OCR failure).

4.2 Backend

4.2.1 FastAPI Service — Query & Agent Orchestration

Handles all synchronous, user-facing request/response traffic from the Streamlit frontend.

- **REST endpoints for:** submitting a chat query, retrieving chat/session history, listing documents and their status, and triggering document deletion.
- **Query Router:** classifies an incoming question as a simple lookup vs a complex/comparative query, and dispatches to the appropriate path.
- **Knowledge-Based RAG path:** embeds the query, performs vector similarity search against the indexed contract chunks, assembles a grounded prompt, and calls the LLM for a direct answer with citations. (Include doc highlighting)

Description

Retrieve relevant contract passages using vector similarity search.

Workflow

User Query
→ Embedding Generation
→ Vector Search
→ Context Retrieval
→ LLM Response

Acceptance Criteria

- Retrieve top relevant chunks.
- Generate grounded responses.
- Provide source citations.
- Support cross-document retrieval.

- **Agentic RAG path:** an orchestrator that can (a) decompose a multi-part question into sub-questions, (b) issue multiple retrieval calls — potentially across different contracts, (c) invoke a comparison/synthesis step, and (d) compose a final answer that references all contributing sources.

- Session and conversation state management (in-memory or persisted) to support follow-up/contextual questions.

- Structured logging of queries, retrieved chunks, and response latency for evaluation and debugging

Description

Use AI agents to plan, reason, retrieve, validate, and synthesize responses.

Agent Workflow

User Query
→ Query Analysis Agent
→ Retrieval Agent
→ Validation Agent

→ Response Generation Agent
→ Final Answer

Example Queries

- Compare payment terms across contracts.
- Which contracts expire within 90 days?
- Summarize termination clauses.
- List all confidentiality obligations.

Acceptance Criteria

- Multi-step reasoning supported.
- Multiple retrieval cycles allowed.
- Citation validation performed.
- Accurate synthesized responses generated.

4.2.2 Azure Functions — Document Ingestion Pipeline

Handles event-driven, asynchronous processing triggered by document uploads, decoupled from the query path so ingestion never blocks chat.

- HTTP-triggered function to accept an uploaded file from the Streamlit app and store it in Azure Blob Storage.

- Blob-triggered function chain that performs: text extraction (including OCR fallback for scanned pages), document chunking (clause-aware where possible), embedding generation, and upsert into the vector store.

- Status updates written to a metadata store (e.g., Azure Table Storage or Cosmos DB) at each pipeline stage, polled by the FastAPI status endpoint / Streamlit UI.

- Dead-letter handling and retry logic for failed extraction or embedding steps, with failure reasons surfaced back to the UI.

- Idempotent processing so re-uploading the same file does not duplicate index entries.

4.2.3 Shared Components

- Vector store for embeddings Pinecone, or FAISS for local/dev) shared by both the ingestion pipeline and the query API.

- LLM provider integration (OpenAI or equivalent) used by both the knowledge-based and agentic RAG paths.

- Centralized configuration and secrets management (Azure Key Vault / environment-based config) for API keys and connection strings.

4.2.4 Guardiels Implementation

Input , Output, LLM

4.3 Non-Functional Requirements

Category	Requirement
Performance	Direct (knowledge-based) queries return in under ~5 seconds; agentic multi-step queries return in under ~20 seconds under normal load.
Accuracy	Responses must cite the source document and section; unsupported claims should be avoided in favor of an explicit “not found in the provided contracts” response.
Scalability	Architecture should support a growing document set via the Functions-based pipeline without re-architecting, even though large-scale load testing is out of scope.

Reliability	Ingestion failures must not crash the chat experience; failed documents are clearly flagged rather than silently dropped.
Observability	Key pipeline stages and query paths are logged for debugging and for capstone evaluation/demo purposes.
Security	Uploaded documents and API keys are stored using standard Azure security practices (Blob access policies, Key Vault); full enterprise access control is a non-goal.

UI Mocks